27882번 - 가희와 지하철역 저장 시스템 2

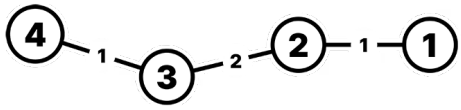
시간 제한	메모리 제한	제출	정답	맞힌 사람	정답 비율
1.5 초	1024 MB	87	24	20	57.143%

문제

가희는 멍멍철도공사에서 관리하는 n 개의 지하철역 정보를 보기 위해, 지하철역 관리 시스템을 운영하고 있습니다. 이 시스템에서, n_1 번 노드에서 n_2 번 노드로 데이터를 보내는 전송 시간은 아래와 같이 계산됩니다.

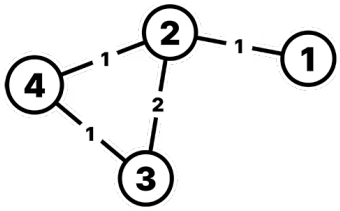
- 데이터가 여러 회선을 통해 전송된 경우, 경로상에 있는 회선들의 전송 시간을 모두 더한 값이 됩니다.
- n_1 번 노드에서 n_2 번 노드로 데이터를 보내는 전송 시간은 n_1 번 노드에서 노드 n_2 번 노드로 가는 경로 중 **총 전송 시간이 가장 작은 값**이 됩니다.

[그림 1]은 4개의 노드로 이루어진 시스템을 나타냅니다.



[그림 1] 노드 4개로 이루어진 시스템

[그림 1]에서 4번 노드에서 1번 노드로 데이터를 보내는 전송 시간은 4입니다. 4번 노드에서 1번 노드로 갈 때, 4번 노드와 3번 노드를 연결하는 회선, 3번 노드와 2번 노드를 연결하는 회선, 2번 노드와 1번 노드를 연결하는 회선의 전송 시간을 모두 더한 값이 4이기 때문입니다. [그림 2]는 [그림 1]의 시스템에서 4번 노드와 2번 노드를 연결하는 회선이 하나 추가된 것입니다.



[그림 2] 그림 1에서 연결 관계가 하나 추가된 시스템

[그림 2]에서 4번 노드에서 1번 노드로 데이터를 보내는 전송 시간은 2입니다. 4번 노드에서 2번 노드를 연결하는 회선, 2번 노드에서 1번 노드를 연결하는 회선을 거치는 것이 총 전송 시간이 가장 적기 때문입니다.

지하철역 저장 시스템은 아래와 같이 동작합니다.

- n 개의 지하철역에 대한 정보는 **하나의 `bucket` 노드**에 저장되어 있습니다.
- 지하철역 s 의 정보를 출력해 달라는 요청이 요청 노드 r 에서 들어온 경우
  - r 에서 **전송 시간이 가장 적은 `cache` 노드**를 찾습니다. 그러한 노드가 여러 개라면, **`id` 가 가장 작은 `cache` 노드**를 찾습니다.
  - 만약, 해당 `cache` 노드에 지하철역 s 에 대한 정보가 있다면, `cache` 노드로부터 정보를 얻어와서 출력합니다.
  - 그렇지 않으면,
    - `cache` 노드 c 가 `bucket` 노드에서 지하철역 s 에 대한 정보를 얻어옵니다.
    - `cache` 노드 c 에 지하철역 s 에 대한 정보를 저장합니다.

지하철역 s 에 대한 정보에 접근했다는 것은 **둘 중 하나를 수행했다는 것을 의미합니다.**

- `cache` 노드 c 에 지하철역 s 에 대한 정보가 있어서, 해당 노드로부터 s 에 대한 정보를 얻어왔습니다.
- `cache` 노드 c 에 지하철역 s 에 대한 정보가 없어서
  - `bucket` 노드에서 지하철역 s 에 대한 정보를 얻어옵니다.
  - 이 정보를 `cache` 노드 c 에 저장합니다.

h **개의 역을 저장할 수 있는 각각의 cache 노드**는 더 이상 지하철역 s 에 대한 정보를 저장할 수 없을 때, 아래와 같이 동작합니다.

- cache 노드 c 가 지하철역 s 에 대한 정보를 저장할 수 없다면, **가장 오랫동안 접근하지 않은 지하철역 정보**를 제거합니다.
- cache 노드 c 에 지하철역 s 에 대한 정보를 저장합니다.

예를 들어 지하철역 3개의 정보를 저장할 수 있는 cache 노드 c_1 이 있다고 해 보겠습니다. 그리고 요청이 아래 [표 1]과 같이 들어왔다고 해 보겠습니다.

시각	요청이 들어온 역
1	OSAKA
2	OSAKA
3	LOOP
4	LINE
5	NOZOMI

[표 1] cache 노드 c_1 에 들어온 요청

시각 4에, LINE 역에 대한 요청을 수행하면 cache 노드 c_1에는 OSAKA , LOOP , LINE 역에 대한 정보가 저장되게 됩니다. 다음 시각 5에 NOZOMI 역에 대한 요청을 수행할 때

- 지하철역 NOZOMI 에 대한 정보가 cache 노드 c_1 에 없습니다.
- 따라서, bucket 노드로부터 NOZOMI 역에 대한 정보를 얻어온 후, cache 노드 c_1 에 저장합니다. 그런데
  - 이미 c_1 에는 지하철역 3개의 정보가 저장되어 있습니다.
  - 따라서, 가장 오랫동안 요청이 들어오지 않은 OSAKA 역에 대한 정보를 cache 노드 c_1 에서 제거합니다.

요청 노드 r 번 노드에서 지하철역 s 에 대한 정보를 출력해 달라는 요청이 수행되는 시간은 아래와 같이 구할 수 있습니다.

- r 번 노드에서 전송 시간이 가장 적은 cache 노드를 찾습니다. 만약에 그러한 것이 여러 개라면, id 가 가장 작은 cache 노드를 찾습니다. 이 노드를 c_1 번 노드라 할 때
  - c_1 번 노드에 지하철역 s 에 대한 정보가 있는 경우 2×(r 번 노드에서 c_1 번 노드로 데이터를 보내는 전송 시간)
  - 그렇지 않은 경우 2×(r 번 노드에서 c_1 번 노드로 데이터를 보내는 전송 시간 + c_1 번 노드에서 bucket 노드로 데이터를 보내는 전송 시간)

지하철역에 대한 정보를 출력해 달라는 요청이 Q 번 주어졌을 때, 각각의 요청이 수행되는 시간을 구해주세요.

입력

첫 번째 줄에 지하철역의 개수 n 과 노드의 개수 m , 문제에서 설명한 h 와 Q 가 공백으로 구분되어 주어집니다.

다음 n 개의 줄에는 한 줄에 하나씩 지하철역 이름이 주어집니다.

다음 m 개의 줄에는 노드에 대한 정보가 아래 포맷으로 주어집니다.

{ node_id } { node_type }

이때 node_id 는 1 이상 10^9 이하의 정수로 주어지며, node_type 은 3개 중 하나로 주어집니다.

- R
  - 요청 노드를 의미합니다.
- C
  - cache 노드를 의미합니다.
- B
  - bucket 노드를 의미합니다.

다음 줄에 노드와 노드의 연결 관계 수 k 가 주어집니다.

다음 k 개의 줄에 아래와 같은 포맷으로 노드와 노드를 연결하는 회선 정보가 주어집니다.

$$\{ node_1 \} \{ node_2 \} \{ r_t \}$$

이는 $node_1$ 와 $node_2$ 를 연결하는 회선이 있고, 이 회선의 전송 시간이 r_t 임을 의미합니다.

다음 Q 개의 줄에 요청에 대한 정보가 아래와 같은 포맷으로 주어집니다.

$$\{ node_1 \} \{ station_name \}$$

이는 id 가 $node_1$ 인 노드에서 역 이름이 $station_name$ 인 역에 대한 정보를 출력해 달라는 요청을 했음을 의미합니다. 이때, $station_name$ 은 **멍멍철도공사에서 관리하는 역 중 하나**로 주어집니다.

출력

요청 하나가 들어올 때마다 각각의 요청이 처리되는 시간을 한 줄에 하나씩 출력해 주세요.

제한

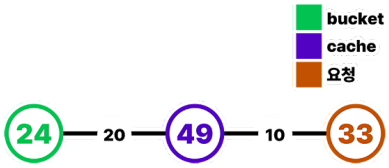
- $1 \le n \le 2 \times 10^5$
- $1 \le m \le 300$
- $1 \le h \le n$
- $1 \le Q \le 2 \times 10^5$
- $1 \le k \le {}_mC_2$
- $1 \le r_t \le 300$
- 역 이름은 길이가 1 이상 10 이하이며, 대소문자와 숫자로만 이루어져 있습니다.
- 멍멍철도공사에서 관리하는 n 개의 역 이름이 중복되는 경우는 없습니다.
- 역에 대한 정보를 출력하라는 요청은 **$cache$ 노드나 $bucket$ 노드에서 이루어지지 않습니다.**
- $bucket$ 노드는 하나만 있으며, $cache$ 노드는 최소 하나 이상 있습니다.
- 서로 다른 노드 번호를 가진 임의의 두 노드를 직접 연결하는 회선은 **0개 또는 1개**입니다.
- $bucket$ 노드로부터 $bucket$ 노드가 아닌 다른 임의의 노드로 **1개 이상의 회선**을 거치면 갈 수 있습니다.

예제 입력 1 복사

```
5 3 2 3
senbatu
ktx
future
abc
next
33 R
49 C
24 B
2
33 49 10
49 24 20
33 future
33 next
33 future
```

예제 출력 1 복사

```
60
60
20
```



[그림 3] 예제 1의 노드 구조

`bucket` 노드는 24번 노드, `cache` 노드는 49번 노드, 요청을 보내는 노드는 33입니다. 매 요청이 들어올 때 마다 아래 알고리즘을 수행합니다.

- 33번 노드로부터 전송 시간이 가장 적은 `cache` 노드인 49번 노드에 역 정보가 있는지를 파악합니다.
- 없으면 24번 노드에서 역 정보를 얻어온 뒤에 `cache` 노드인 49에 역 정보를 저장합니다.

3개의 요청을 날렸을 때, 소요 시간은 아래와 같이 계산됩니다.

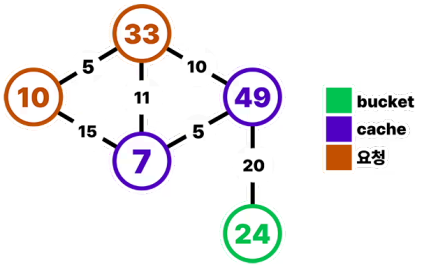
- 1번째 요청을 날렸을 때, 49번 노드에는 아무런 역에 대한 정보도 저장되어 있지 않습니다.
  - 따라서 24번 노드인 `bucket` 노드에서 `future` 역에 대한 정보를 가져옵니다.
  - 다음에 49번 노드에 `future` 역에 대한 정보를 저장합니다.
  - 이 때 요청에 걸리는 시간은 `2×(10+20)` 이 됩니다.
- 2번째 요청을 날렸을 때, 49번 노드에는 `future` 역에 대한 정보가 저장되어 있습니다. `next` 역에 대한 정보는 저장되어 있지 않으므로
  - 24번 노드인 `bucket` 노드에서 `next` 역에 대한 정보를 가져옵니다.
  - 다음에 49번 노드에 `next` 역에 대한 정보를 저장합니다.
  - 이 때 요청에 걸리는 시간은 `2×(10+20)` 이 됩니다.
- 3번째 요청을 날렸을 때, 49번 노드에는 `future` 역과 `next` 역에 대한 정보가 저장되어 있습니다.
  - 그렇기 때문에, 24번 노드에서 정보를 얻어옵니다. 요청에 걸리는 시간은 `2×10` 이 됩니다.

예제 입력 2 복사

```
7 5 2 4
sindorim
sadang
jamsil
hongikuniv
gangnam
future
next
33 R
49 C
24 B
7 C
10 R
6
24 49 20
49 33 10
49 7 5
33 7 11
33 10 5
10 7 15
33 future
33 next
33 future
10 future
```

예제 출력 2 복사

```
60
60
20
80
```



[그림 4] 예제 2의 노드 구조

먼저 33번 노드와 10번 노드로부터 가장 전송 시간이 적은 `cache` 노드는 [표 2]와 같습니다.

기준 노드	기준 노드로부터 전송 시간이 가장 적은 cache 노드	전송 시간
33	**49**	10
10	49. **7**	15

[표 2] 노드 33, 노드 10으로부터 가장 전송 시간이 적은 `cache` 노드

노드 10으로부터 가장 전송 시간이 적은 `cache` 노드는 여러 개가 있습니다. 이 중 id가 가장 작은 것은 7번이므로, 10번 노드는 `cache` 노드로 7번 노드를 이용합니다.

4번째 요청은 10번 노드에서 `future` 역에 대한 정보를 얻어오라는 요청입니다. 10번 노드에서는 `cache` 노드로 7번 노드를 쓰게 됩니다.

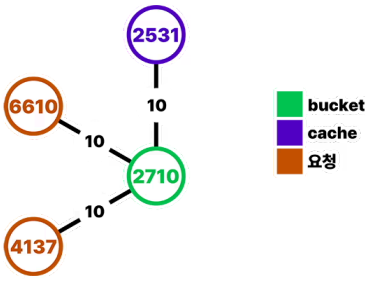
- 4번째 요청을 날렸을 때, 7번 노드에는 역에 대한 정보가 저장되어 있지 않습니다. 따라서
  - 24번 노드인 `bucket` 노드에서 `future` 역에 대한 정보를 가져옵니다.
  - 다음에 7번 노드에 `future` 역에 대한 정보를 저장합니다.
  - 이 때 요청에 걸리는 시간은 `2×(15+25)` 이 됩니다.

예제 입력 3 복사

```
5 4 2 5
eeeah
subway
kono
akamada
na
4137 R
2531 C
6610 R
2710 B
3
2531 2710 10
2710 6610 10
2710 4137 10
6610 kono
6610 kono
4137 akamada
6610 subway
4137 akamada
```

예제 출력 3 복사

```
60
40
60
60
40
```



[그림 5] 예제 3의 노드 구조

먼저 6610번 노드에서 kono 역에 대한 정보를 얻어오라는 요청이 들어왔습니다. 6610번 노드로부터 가장 전송 시간이 적은 cache 노드는 2531입니다. 처음에는 cache 노드 2710에 kono 역에 대한 정보가 없습니다. 따라서

- 요청을 날린 6610번 노드에서 가장 전송 시간이 적은 cache 노드 2531번 노드까지 데이터 전송 시간 20
- cache 노드 2531번 노드에서 bucket 노드 2710번 노드까지 데이터 전송 시간 10

이 둘을 합한 것의 2를 곱한 60이 답이 됩니다.

그 이후에 다시 kono 역에 대한 정보를 출력하라는 요청이 6610번 노드에서 들어왔습니다. 이 때에는 cache 노드 2710에 kono 역에 대한 정보가 있습니다. 따라서

- 요청을 날린 6610번 노드에서 가장 전송 시간이 적은 cache 노드 2531번 노드까지 데이터 전송 시간 20

에 2를 곱한 40이 답이 됩니다.

다음에 4137번 노드에서 akamada 역에 대한 정보를 출력하라는 요청이 들어왔습니다. cache 노드 2710에 akamada 역에 대한 정보는 없으므로

- 요청을 날린 4137번 노드에서 가장 전송 시간이 적은 cache 노드 2531번 노드까지 데이터 전송 시간 20
- cache 노드 2531번 노드에서 bucket 노드인 2710번 노드까지 데이터 전송 시간 10

에 2를 곱한 60이 답이 됩니다. 이 때, 2710번 `cache` 노드에 저장된 역에 대한 정보와 마지막으로 접근한 시각은 [표 3]과 같습니다.

접근한 시각	역
2	kono
3	akamada

[표 3] 3번째 요청까지 수행된 후 `cache` 노드 2710에 저장되어 있는 역 정보

4번째는 6610번 노드에서 `subway` 역에 대한 정보를 출력하라는 요청입니다. `cache` 노드 2710에 `subway` 역에 대한 정보는 없으므로, 답은 60이 됩니다. 그리고, [표 3]에 따르면, 더 이상 더 이상 역을 저장할 수 없습니다. 따라서

- 가장 오랫동안 접근하지 않은 `kono` 역에 대한 정보를 `cache` 노드에서 제거합니다.
- `subway` 역에 대한 정보를 추가합니다.

접근한 시각	역
4	subway
3	akamada

[표 4] 4번째 요청까지 수행된 후 `cache` 노드 2710에 저장되어 있는 역 정보

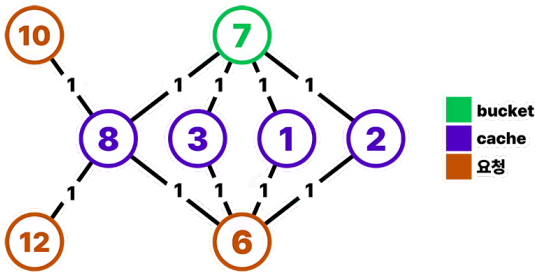
5번째는 4137번 노드에서 `akamada` 역에 대한 정보를 출력하라는 요청입니다. `cache` 노드 2710에 `akamada` 역에 대한 정보는 있으므로, 답은 40이 됩니다.

예제 입력 4 복사

```
7 8 2 10
Hyou
cKan
doT
mor4e
anyt8hing
romantic
byoe
6 R
2 C
1 C
3 C
7 B
8 C
12 R
10 R
10
6 2 1
6 1 1
6 3 1
6 8 1
2 7 1
1 7 1
7 3 1
8 7 1
10 8 1
8 12 1
6 romantic
12 byoe
10 romantic
12 anyt8hing
10 byoe
12 romantic
10 byoe
6 cKan
10 mor4e
6 anyt8hing
```

예제 출력 4 복사

```
4
4
4
4
4
4
2
4
4
4
```



[그림 6] 예제 4의 노드 구조

노드 6, 10, 12로부터 가장 전송 시간이 적은 `cache` 노드는 [표 5]와 같습니다. 이 때 각 노드가 쓰는 `cache` 노드는 굵게 표시되어 있습니다.

기준 노드	기준 노드로부터 전송 시간이 가장 적은 `cache` 노드	전송 시간
6	**1**, 2, 3, 8	1
10, 12	**8**	1

[표 5] 노드 6, 노드 10, 노드 12로부터 가장 전송 시간이 적은 `cache` 노드

6번 요청 노드에서 역에 대한 정보를 얻어오는 요청은 1번 `cache` 노드를 거치게 됩니다. 이 때 `romantic`, `cKan`, `anyt8hing` 순서대로 역에 대한 정보가 있는지 검사하게 됩니다. 1번 `cache` 노드 입장에서, **항상 새로운 역에 대한 요청을 날리기 때문에**, `bucket` 노드에서 정보를 얻어오게 됩니다.

10번과 12번 요청 노드에서 역에 대한 정보를 얻어오는 요청은 8번 `cache` 노드를 거치게 됩니다. 이 때 `byoe`, `romantic`, `anyt8hing`, `byoe`, `romantic`, `byoe`, `mor4e` 순서대로 역에 대한 정보가 있는지 검사하게 됩니다.

시간	오래 전에 접근	최근에 접근	요청	`cache` 노드에 있었는지 여부
2			byoe	X
...	...	...	...	...
6	anyt8hing	byoe	romantic	X
7	**byoe**	romantic	byoe	O
8	romantic	byoe	mor4e	X
...	...	...	...	...

[표 6] 8번 `cache` 노드에 저장되어 있는 역 정보

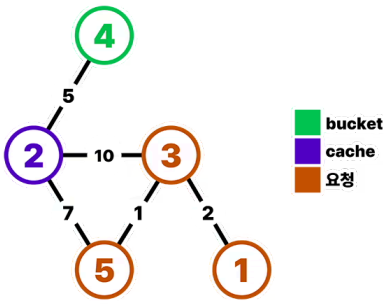
이 때, 8번 `cache` 노드에는 7번째 요청 `byoe` 역에 대한 정보를 얻어올 때에만 `cache` 노드에서 정보를 얻어오게 됩니다. **따라서 7번째 요청에 대한 답만 2가 됩니다.**

예제 입력 5 복사

```
7 5 2 5
honizzang
nuotameno
zezkkyo
4daie
machQine
party
dontstop
1 R
3 R
5 R
2 C
4 B
5
1 3 2
3 5 1
5 2 7
3 2 10
2 4 5
1 party
1 dontstop
1 honizzang
1 dontstop
1 party
```

예제 출력 5 복사

```
30
30
30
20
30
```



[그림 7] 예제 5의 노드 구조

예제 입력 6 복사

```
3 3 2 5
Pareo
Wa
Emerald
1 R
2 C
3 B
2
1 2 1
2 3 10
1 Pareo
1 Wa
1 Pareo
1 Emerald
1 Pareo
```

예제 출력 6 복사

```
22
22
2
22
2
```

예제 입력 7 복사

```
5 3 1 5
then
wrong
time
no
see
33 R
49 C
24 B
2
33 49 10
49 24 20
33 then
33 wrong
33 then
33 then
33 then
```

예제 출력 7 복사

```
60
60
60
20
20
```

출처

Contest (/category/45) > BOJ User Contest (/category/984) > 가희와 함께 하는 코딩 테스트 (/category/529) > 가희와 함께 하는 5회 코딩테스트 (/category/detail/3565) G번

- 문제를 만든 사람: chogahui05 (/user/chogahui05)
- 문제를 검수한 사람: chansol (/user/chansol), egod1537 (/user/egod1537), kiwiyou (/user/kiwiyou), utilforever (/user/utilforever)